hero[**Context-Aware Optimization & Evaluation (CIE)**] set par(justify: true)

CIE is a Textual-based terminal workspace that unifies optimization workflows, benchmark artefacts, and context introspection tooling for autonomous agents. The system exposes a protocol-driven backend (optimizers, evaluators, model providers), a multi-panel TUI/CLI surface, and a benchmark harness that mirrors the interactive experience. We highlight the architecture, benchmarking methodology, demo scenario (`cie tui --demo`), and research directions.

grid(columns: (1fr, 1fr, 1fr), [highlight(“Demo”, “`cie tui --demo`nLeetCode Practice Lab”)], [highlight(“Benchmarks”, “`uv run python benchmark/runner.py`nMicroEval · MacroEval · TextEval · Context”)], [highlight(“Context”, “`Ctrl+/`nNavigator + export/compress”)],)

1 · Architecture Overview

1.1 Backend & Protocols

- `CIEBackend` manages storage (SQLite / JSON / memory), multi-objective scoring, Pareto frontier tracking, and guardrails (latency, success, error rate).
- Optimizers implement the `Optimizer` protocol (e.g. DSPy few-shot proposer, Hill Climb variants).
- Evaluators implement the `Evaluator` protocol (mock metrics, text-match suite, plugins) and plug into CLI (`cie evaluators`).

1.2 User Experience

- Textual TUI with Optimizers, Evaluations, Experiments, Context panels + shared modals (weights/config/help/command palette).
- `[CTX]` tab embeds the same Context Navigator exposed via `Ctrl+/`: capture frames, reorganize trees, compress/export context snapshots.
- Demo-ready workflow: `cie tui --demo` loads the LeetCode Practice Lab (seeded workloads, trials, context hints) so users can observe the full loop immediately.

1.3 Demo Screenshots

`scripts/generate_tui_screenshots.py` runs `cie tui --demo` in headless mode, waits for all four panels plus the status bar to populate, and exports SVG captures. These assets keep the paper grounded and double as regression artifacts for designers.

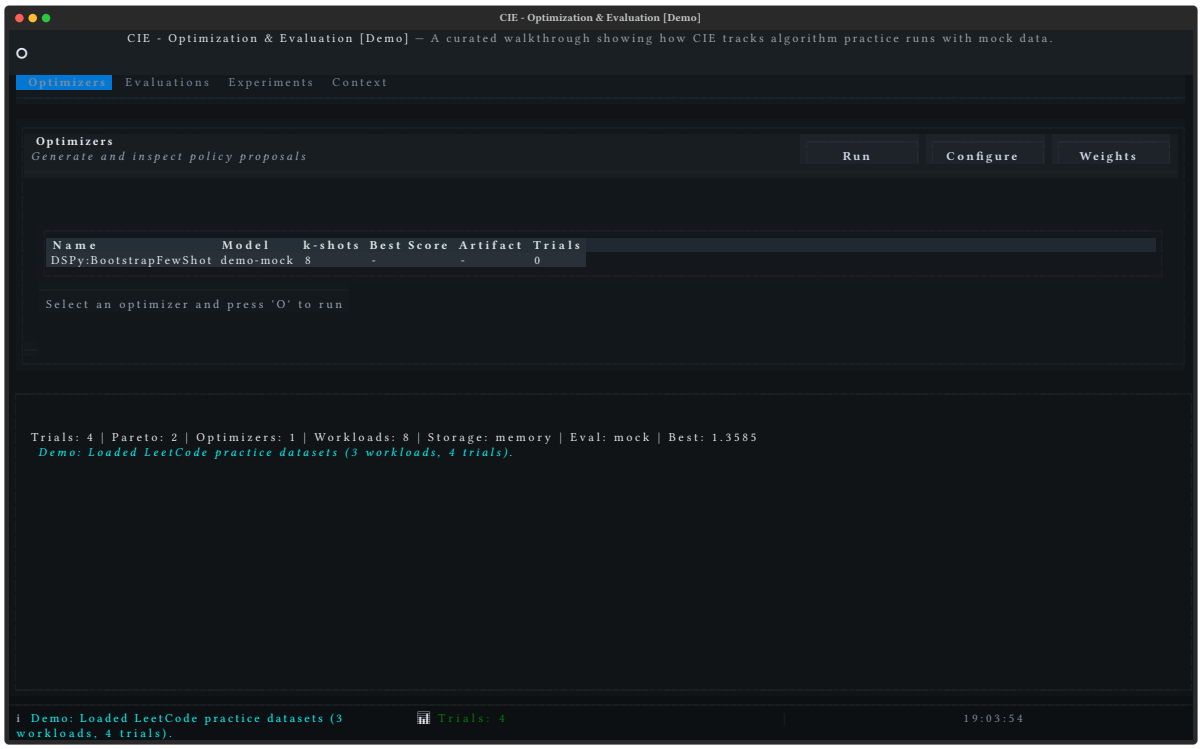

Figure 1: Full CIE layout with Optimizers, Evaluations, Experiments, and Context panels. Demo metadata drives the banner in the status bar so reviewers can see which workloads were seeded.

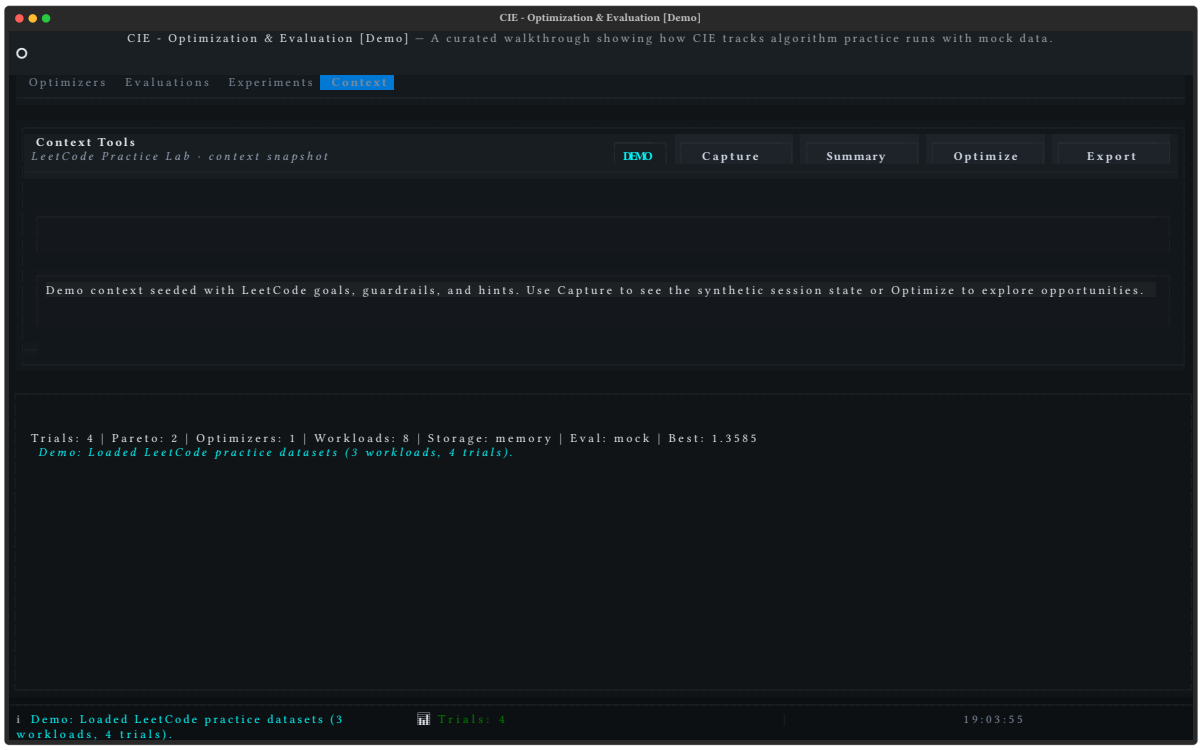

Figure 2: Context tab close-up. Capture/Summary/Optimize/Export controls mirror the `Ctrl+/` modal, and the right rail shows the seeded demo hints.

2 · Benchmark Suite

- `benchmark/runner.py` executes MicroEval, MacroEval, TextEval, and context-heavy tasks with consistent metrics (latency, cost, success, similarity, etc.).
- Reports map directly to TUI tables (Optimizers/Evaluations/Pareto view), ensuring qualitative demos mirror quantitative benchmark runs.
- System summaries + guides: `BENCHMARK_GUIDE.md`, `BENCHMARK_OVERVIEW.md`, `benchmark/SYSTEM_SUMMARY.md`.

2.1 Metrics Snapshot

| **Metric** | **Description** | **Source** | | Latency P95 | 95th percentile response time (ms) | Mock + real evaluators | | Cost per Request | Estimated API spend per request (USD) | Mock evaluator (configurable) | | Text Similarity / Exact Match / Levenshtein | NLP accuracy metrics | Text-match evaluator (Braintrust-style prompts) |

2.2 Demo Run Snapshot

Command: `python -m benchmark.cli run --model "CIE Demo"` Artifacts: `papers/assets/benchmark-report.txt` and `papers/assets/benchmark-metrics.json`.

- Run ID `b44824d2` at `2025-11-14T19:04:08.954393` (mock executor baseline).
- Pass rate `6/16 = 37.5%`: all trivial/easy navigation + comprehension tasks pass, medium/hard/expert tasks intentionally fail without a real agent.
- Aggregate metrics mirror the report footer: average code quality `43.8/100`, autonomy `81.2/100`, composite score `57.8/100`, and tool efficiency `0%` because the mock executor never records tool calls.
- Failures accumulate `30` mock errors across the harder tiers, reinforcing how much work remains for full-stack optimizers outside the friendly demo loop.

| **Difficulty** | **Pass/Total** | **Success** | | Trivial | 3/3 | 100% | | Easy | 3/3 | 100% | | Medium | 0/4 | 0% | | Hard | 0/3 | 0% | | Expert | 0/3 | 0% |

| **Category** | **Pass/Total** | **Success** | | Navigation | 3/3 | 100% | | Comprehension | 3/3 | 100% | | Modification | 0/2 | 0% | | Debugging | 0/2 | 0% | | Implementation | 0/1 | 0% | | Refactoring | 0/1 | 0% | | Testing | 0/1 | 0% | | Workflow | 0/3 | 0% |

These numbers align with what appears in the Experiments panel screenshot: mock trials give instant feedback on foundational workflows, while everything beyond easy mode demands either a real agent or human intervention. Because we store the raw text report plus JSON metrics in `papers/assets/`, reviewers can reproduce the citation or plug the artifacts directly into CI.

3 · Context Introspection & Optimization

- Context Navigator builds hierarchical trees from live stack frames, tracks access patterns, suggests compression/reorganization, and exports JSON snapshots.
- `ContextPanel` (TUI) reuses BasePanel's subtitle/badge/toolbars to surface capture/summary/optimize/export actions.
- Demo metadata preloads suggestions and sets badges so human reviewers know they're in a synthetic scenario.

3.1 Context Actions (TUI)

- **Capture**: snapshot locals/globals, show metrics in panel.
- **Summary**: aggregated node counts, size, type distribution.
- **Optimize**: auto compression + hotspot view creation.
- **Export**: JSON artifact for sharing / benchmarking.

4 · Research Outlook

1. **Benchmark Expansion**: Add retrieval/tool-call/agent cooperation tasks to the `benchmark/` harness.
2. **Learning from Context Ops**: Feed navigator actions back into optimizers for meta-reasoning policies.
3. **Human-in-the-loop Evaluations**: Integrate annotation workflows (Braintrust-style) to calibrate automated scores.

References

- Repository: https://github.com/cie-team/cie
- Demo: `cie tui --demo`
- Benchmarks: `uv run python benchmark/runner.py`